HostelOS: Presentation Speech

BTech CSE Final Year Project Evaluation. Team: Gokul SR, Jeen George, S Adithyan, Albi Thomas. Guide: Manolekshmi I. Coordinator: Dr. Jayakrishnan R.

A slide-by-slide speech in simple English, about 15 to 18 minutes at a relaxed pace.

Part 1: Opening

Slide 1: Title

Good morning, respected panel members, our project guide, and our project coordinator. We are the final-year B.Tech CSE team: Gokul, Jeen, Adithyan and Albi. Today we are presenting our project, **HostelOS: An Intelligent Hostel Operations, Safety and Resource Optimization Platform**. I will take you through the problem, our solution, the design, and the screens we have built so far.

Slide 2: Abstract

In one line, HostelOS is one platform that brings together everything a hostel does every day. That includes administration, student safety, complaints, maintenance, payments and analytics. It has three main ideas. First, all records are connected, instead of being kept in separate places. Second, QR codes are used only for special movements, like late returns or outings, so normal daily movement stays easy. Third, we propose AI help for things like complaint sorting and forecasting. We want to be clear that we are not claiming any accuracy results yet.

Slide 3: Introduction

A hostel is a busy place. Every day it handles rooms, leave, gate movement, complaints, repairs, bookings, payments and sometimes emergencies. These things are connected. For example, when a student goes on leave, it changes their presence status. When a complaint comes in, it may need a maintenance team. When records are kept separately, staff repeat the same work, and decisions become slow and hard to trace. HostelOS is built to give one connected view of all this.

Slide 4: Literature Survey, 2024 to 2026

We studied recent research first. Alsawi and Ibrahim reviewed QR-based access control, but that is a survey and not a working system. Yu and team used machine learning to sort complaints, but with healthcare data, not hostels. One IEEE paper used IoT sensors for energy, safety and theft prevention, but it depends heavily on hardware and has no single

admin dashboard. Adedoyin, Olukoya and Tan worked on QR smart locks and full-stack administration, but with limited intelligence across areas. So recent work is strong in single areas, but not in one complete hostel view.

Slide 5: Literature Survey, 2018 to 2023

Older studies gave us the foundations. Jain and team built QR door access, but without any leave or presence status behind it. Bhardwaj and team used face recognition and IoT for visitors, which needs heavy hardware and raises privacy concerns. Singh and team used RFID and sensors for attendance and room power control, but did not cover complaints, mess or safety. Patel and Okokpujie worked on QR, RFID and biometric systems, which need special hardware or create daily friction. So access, attendance and IoT are useful parts, but not a full operating model.

Slide 6: Existing System

In most hostels today, management is digital only in pieces. Rooms, fees, leave and complaints are handled separately, often on paper. Some places add hardware like RFID or cameras. The data exists, but it does not connect, so it cannot be used for prediction or planning. The result is a lot of recorded activity, but no clear overall picture.

Slide 7: Drawbacks and Research Gap

We found four main drawbacks. One, **fragmentation**: separate modules make it hard to decide across workflows. Two, **manual work**: repeated data entry and handoffs take extra effort. Three, **limited intelligence**: current systems give very little prediction or decision support. Four, **access-control friction**: special hardware adds cost, maintenance and daily inconvenience. So our research gap is a single, auditable hostel platform that combines administration, safety, predictive analytics and resource optimization.

Slide 8: Problem Statement

Our problem statement is simple. Hostel operations do not have one integrated, intelligent and auditable platform that connects administration, resources and student safety. It should also keep routine movement practical, and make sure exceptional movement is properly authorized. HostelOS tries to fill the gap between basic hostel software and a connected system. We also want to be honest in this project about what is proposed, what is built, and what has been measured.

Slide 9: Proposed Solution

Our solution connects operations, safety and insight in four parts. **Administration** covers students, rooms and beds, leave, presence and financial records. **Safety** covers QR authorization for exceptional movement, server-side checking, SOS, and an audit trail.

Resident services covers complaints, maintenance, mess, inventory and amenity bookings. **Analytics** covers proposed complaint classification, predictive maintenance, forecasting and risk analysis. Next, I will move into the system design.

Slide 10: Objectives

We have five objectives. First, role-based administration, including presence tracking and movement authorization. Second, explainable room and amenity allocation, like Wi-Fi, gym and recreation booking. Third, guided complaints with self-checks, and proposed priority routing. Fourth, planning support such as predictive maintenance, meal and inventory forecasting, and SOS tracking. Fifth, careful analytics, where we do not claim accuracy or deployment that we have not verified. These objectives drive the design in the next section.

Part 2: System Design

Slide 11: System Architecture

HostelOS uses a layered design, which keeps each part clean and separate. The **presentation layer** is what users see, and it talks to the backend through APIs. The **application layer** has the business logic, login, and role-based access. The **intelligence layer** holds the proposed classification, forecasting, allocation and risk analysis. The **data layer** is one central database for all records. The **notification and security layer** handles alerts, authorization and verification.

Slide 12: Hardware and Software Requirements

For the server, we need an application host with PostgreSQL storage on Linux. For clients, students and staff use smartphones, and wardens and admins use PCs or laptops. The phone camera does the QR scanning, so **no special RFID hardware is needed**. Our proposed stack is React, FastAPI or Express, PostgreSQL, Flutter or React Native, JWT with RBAC, scikit-learn for ML, and Git and Docker. These are selected or proposed technologies, not final implementation results.

Slide 13: Entity Relationship Diagram

This is our data model. The **Student** is the central record, and everything links to it. A student links to a room, leave requests, complaints, movement records, SOS incidents, payments and bookings. A leave request connects to QR tokens, which are only for approved exceptional movement. Primary keys and foreign keys keep all these links clear and traceable.

Slide 14: Database Design and Auditability

We can group the data into three areas. **Identity and accommodation** holds student and room records. **Movement** holds leave requests, QR tokens and movement records, so we can trace approval, expiry, gate check and result. **Operations** holds complaints, SOS incidents, payments and bookings. Because identity, movement and services stay connected, every action can be traced later.

Slide 15: Exceptional Movement (QR Workflow)

This is our most important workflow. First, ordinary movement on normal working days needs **no QR**. For a special case, the student requests exceptional movement. The backend notifies the warden, who approves or rejects. If approved, a temporary QR token is issued. At the gate, the student shows the QR and security scans it. The **server** then checks the token, expiry, student, permission and state. Finally, the outcome is recorded with an audit trail. Invalid, expired or reused tokens are rejected.

Slide 16: Module 1, Identity and Daily Administration

Our work is divided into five modules. Module 1 covers **core administration and identity**. It includes login with role-based access, student records and accommodation, room and bed management with explainable allocation, and presence, leave and exceptional-movement authorization.

Slide 17: Module 2, Safety Workflows

Module 2 is **safety and emergency**. It covers SOS reporting, acknowledgement and response tracking. It also covers security workflows and the server-side QR check, safety mapping for fire and first-aid points where documented, and pre-complaint safety self-checks such as checking the MCB or a valve. Everything here is meant to be traceable.

Slide 18: Module 3, Intelligence and Analytics

Module 3 is **intelligence and analytics**. It proposes complaint classification and priority routing, predictive maintenance from past records, meal and inventory forecasting with waste analysis, and risk analysis for administrators. These are proposed capabilities. We are not claiming any training, deployment or accuracy results.

Slide 19: Module 4, Resident Services and Finance

Module 4 covers **resident services and finance**. This includes mess and resident-facing services, facility bookings, subscriptions and amenity access, fees, invoices and payments, and notifications, along with the public showcase. These connect everyday student services to their account activity.

Slide 20: Module 5, Resource and Facility Management

Module 5 is **resource and facility utilization**. It includes maintenance coordination, inventory and shared amenity management, and capacity planning for rooms, facilities and services. With this, the system design and the five-module overview are complete. Part 3 shows user roles and real screenshots.

Part 3: Roles and Screens

Slide 21: User Roles and Permissions

HostelOS has six user types, and each one sees only what they need. The **student** handles passes, room issues, bookings and fees. The **warden** handles approvals, attendance, complaints and SOS monitoring. The **hostel administrator** handles records, allocation, oversight and analytics. The **parent or guardian** sees the student's safety status and approved outings. **Gate security** scans QR codes and verifies movement. **Mess and maintenance staff** do their own service and repair work.

Slide 22: Role-Based Workflow

Access is controlled in simple steps. The user logs in, and JWT with RBAC decides which dashboard and actions they get. They perform an allowed action, a decision is received or recorded, and the operational state is updated. The backend APIs sit in the middle, connecting the clients, the business rules and the PostgreSQL data.

Slide 23: Login and Role Selection

Now the real screens. First, the user chooses a role. Then they see the sign-in page for that role. Here you can see the Student Portal sign-in and the Hostel Warden sign-in.

Slide 24: Student View

This is the student dashboard. It shows a quick overview in one place: the student's room, A-102 in this example, the current presence status as "inside hostel", gate-pass status, Wi-Fi and emergency contact information. These are interface screens, not proof of the full backend behaviour.

Slide 25: Gate-Pass Workflow

Here the student requests a gate pass. They choose the pass type, time, destination and reason. The warden reviews it and records the approval or rejection. If it is approved, a temporary QR token can be issued for the gate check later.

Slide 26: Warden Operations

The warden gets a monitoring view, with three states: **Inside**, **Out on Pass** and **On Leave**. The warden's page also has tabs for pass requests, emergency alerts and room complaints, so the warden can handle everything from one place. These states are examples taken from our screenshots.

Slide 27: Complaint Handling

Complaints go from guided intake to visible resolution. The student registers a complaint by choosing a category, following the self-check guidance and adding a description. The warden then manages it in the room-complaint view, and the status stays visible until it is resolved. This screen shows complaint handling. It does not mean the AI classifier is already working.

Slide 28: Bookings and Fees

Everyday services are also on the platform. Students can book gym and sports slots by choosing a time. They can also see fee categories and invoices, with payment-status labels. These labels are interface states, not proof of real live payments.

Slide 29: Safety Visibility

Safety is visible to the right people. The **parent or guardian** can see the student's presence, approved outings and helpline numbers. The **warden** can see an SOS incident's location and its response status. This gives families peace of mind and helps staff respond faster.

Closing

Slide 30: Conclusion, Future Scope and References

To conclude, HostelOS is one proposed platform that brings together hostel administration, role-based access, QR authorization only for exceptional movement, resident services, safety workflows and AI-assisted decision support.

For **future scope**, we plan to validate real deployment, complete and test the intelligence pipelines, deepen the analytics, extend integrations, and confirm the module boundaries against the final project documentation.

For **limitations**, the screenshots show interface states only. They do not prove live transactions, trained AI performance or production deployment.

Our references are listed on this slide, covering the studies we reviewed from 2018 to 2026.

Thank you all for listening. We would be happy to answer your questions.

Delivery Tips

- **Split by team member:** a natural split is Gokul for slides 1 to 10, Jeen for 11 to 15, Adithyan for 16 to 22, and Albi for 23 to 30.
- **Pace:** if you have less time, shorten slides 4 and 5 (just name the gap) and slides 16 to 20 (one line per module).
- **Likely questions:** "Is the AI working?" Answer: no, it is proposed, and we are not claiming results. "Why QR only for exceptions?" Answer: to avoid daily friction and hardware cost.
- **Check before presenting:** the slide text was read using text recognition on images, so verify names such as Okokpujie and Adedoyin against your original deck.